HDOC Day-61

****

Write a C program that reads the size and elements of a variable-length integer array, nums[], containing only positive integers, and an integer target. Pass the array to a function named two_sum() using its address. Use pointers to pass the array between the calling function and two_sum(). The function two_sum() must:

- Find two distinct indices, i and j, such that: nums[i] + nums[j] = target.
- Display the two indices separated by a space.
- Return the indices in any order.
- Display -1 -1 if no such pair exists.
- Use a nested-loop approach to solve the problem.
- Assume that at most one valid solution exists.

1. Problem Understanding

The program reads the size of a variable-length integer array, its positive integer elements, and a target value. The array is passed to two_sum() using its address. The function compares every pair of distinct elements using nested loops. If a pair whose sum equals the target is found, the corresponding indices are displayed. If no valid pair exists, -1 -1 is displayed.

2. Standard Test Cases

Test Case	Size (n)	Array Input	Target	Expected Output
1	4	2 7 11 15	9	0 1
2	3	3 2 4	6	1 2
3	2	3 3	6	0 1
4	3	1 2 3	7	-1 -1
5	5	5 1 9 4 7	11	3 4

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare a variable-length integer array nums[n].
4. Read n positive integer elements into nums[].
5. Read the target value.
6. Pass nums, n, and target to two_sum() using the address of the array.
7. Stop.

Function: two_sum()

1. Use an outer loop i from 0 to n - 2.
2. Use an inner loop j from i + 1 to n - 1.
3. Check whether *(nums + i) + *(nums + j) equals target.

4. If equal, display i and j and return from the function.
5. If all pairs are checked and no pair is found, display -1 -1.

4. C Program

```
#include <stdio.h>

void two_sum(int *nums, int n, int target)
{
   int i, j;

   for(i = 0; i < n - 1; i++)
   {
      for(j = i + 1; j < n; j++)
      {
         if(*(nums + i) + *(nums + j) == target)
         {
            printf("%d %d\n", i, j);
            return;
         }
      }
   }

   printf("-1 -1\n");
}

int main()
{
   int n, i, target;

   printf("Enter the size of array: ");
   scanf("%d", &n);

   int nums[n];

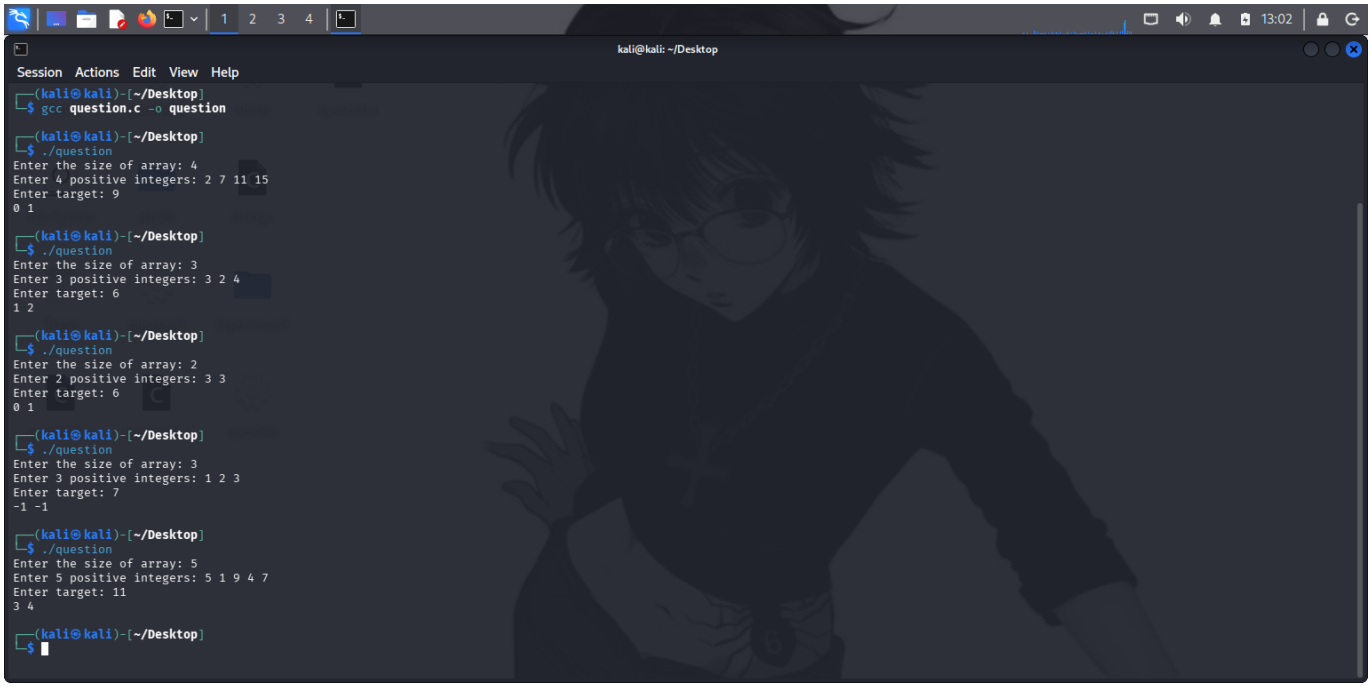
   printf("Enter %d positive integers: ", n);
   for(i = 0; i < n; i++)
      scanf("%d", &nums[i]);

   printf("Enter target: ");
   scanf("%d", &target);

   two_sum(nums, n, target);

   return 0;
}
```

5. Compilation and Execution Evidence



6. Verification

Test Case	Size (n)	Array Input	Target	Actual Output	Result
1	4	2 7 11 15	9	0 1	Pass
2	3	3 2 4	6	1 2	Pass
3	2	3 3	6	0 1	Pass
4	3	1 2 3	7	-1 -1	Pass
5	5	5 1 9 4 7	11	3 4	Pass